



Search



Write



★ Member-only story

Understanding YOLOv7 Neural Network

A bit more detailed ...



Nahid Alam · [Follow](#)

11 min read · Jun 4, 2023



133



5



Note: This is a living document. Expect it to get updated as I dig more.

1. Introduction

YOLOv7 is one of the models in the YOLO (You Only Look Once) series of object detection. There are many articles on the web that discusses YOLOv7 architecture. But none of them are comprehensive enough with end-to-end

architectural component description. The purpose of this post is to serve as a guide for end-to-end YOLOv7 neural network understanding.

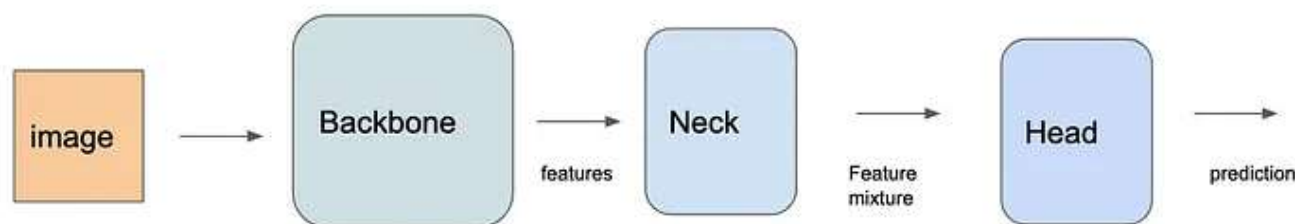
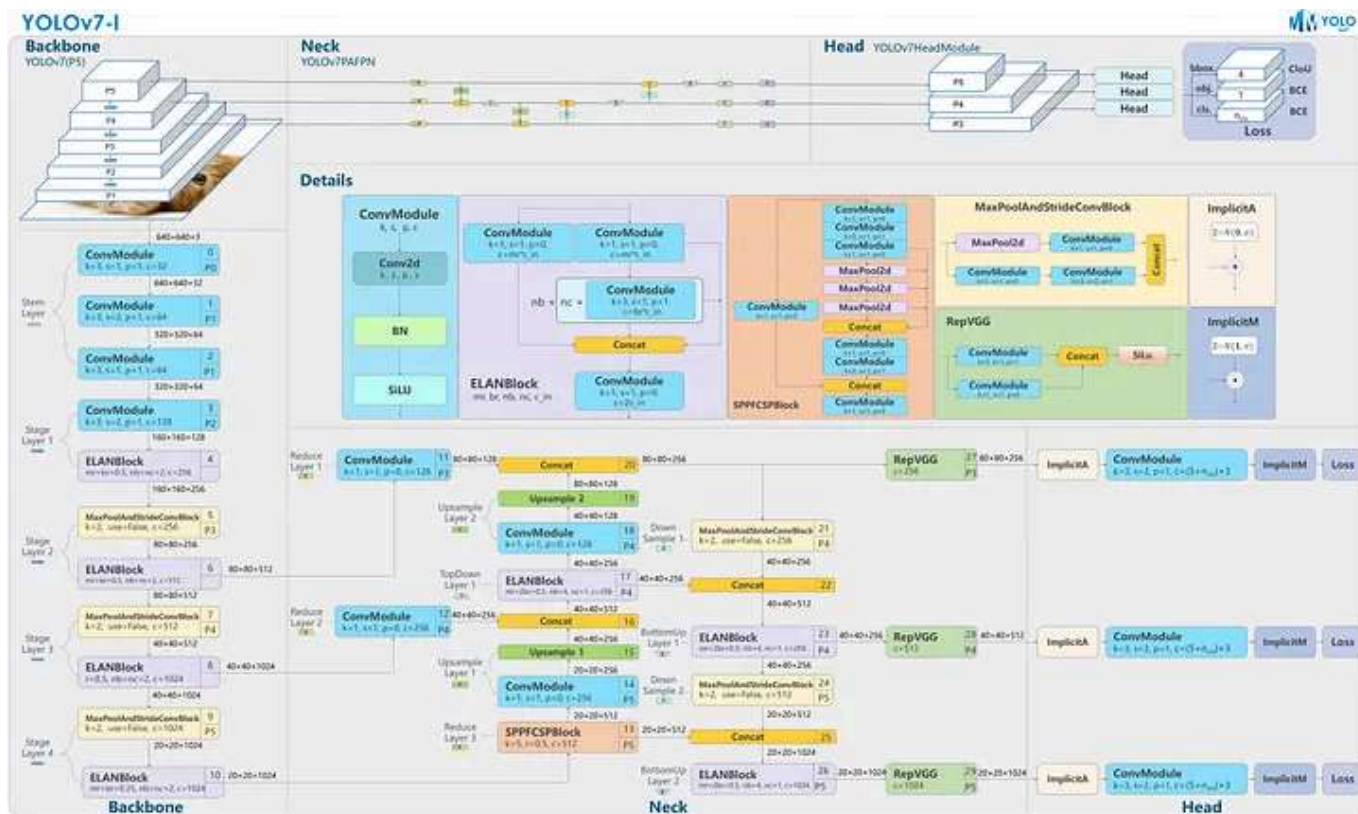


Figure 1: General YOLO architecture at a high level

YOLO network consists of three main components as shown in Figure 1

1. **Backbone:** A convolutional neural network creates images features aka. embeddings
2. **Neck:** A collection of neural network layers that combines and mixes features to pass it to the next stage for prediction
3. **Head:** Consumes features from the neck creates prediction outputs.

Specifically YOLOv7 architecture looks like below

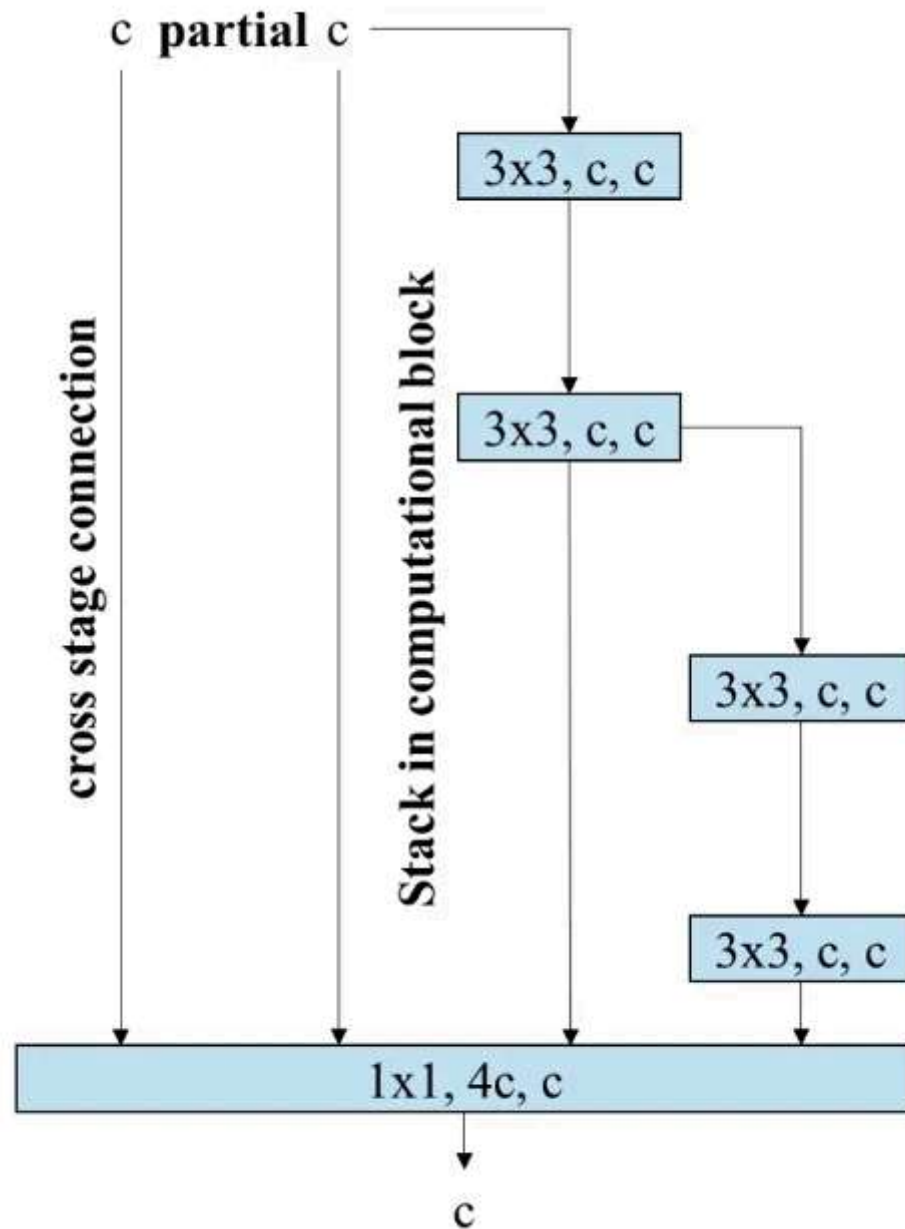
Figure 2: Detailed YOLOv7 architecture [Source](#)

Note that the diagram on Figure 2 was created by the folks at mmlab, not by the authors. Therefore naming of some network blocks might not exactly match with the YOLOv7 paper. For example, ELANBlock in Figure 2 refers to the E-ELAN module mentioned in the paper which we are going to discuss now.

2. What is new in YOLOv7

2.1 Extended-Efficient Layer Aggregation Network (E-ELAN)

E-ELAN extends the concept of ELAN

Figure 3: Collected from ELAN [paper](#)

2.1.1 Why was ELAN born?

The purpose of ELAN is to solve the problem that the convergence of the deep model will gradually deteriorate when executing model scaling.

- When the network reaches a certain critical depth — 1) if we continue to stack computational blocks, the accuracy gain will be less and less; 2) its convergence begins to deteriorate, resulting in an overall accuracy that is worse than shallow networks.
- For example, ResNet-152 is about three times as computationally intensive as ResNet-50, but offers less than 1% improvement in accuracy on ImageNet. When ResNet is stacked to 200 layers, its accuracy is even worse than ResNet-152.

ELAN authors are trying to solve this problem by controlling the shortest gradient path so that it does not quickly become longer.

In YOLOv7, the authors propose Extended-ELAN (E-ELAN) as shown in the diagram below. At E-ELAN, gradient flow path remains the same as ELAN. Only the cardinality increases through the use of group convolution.

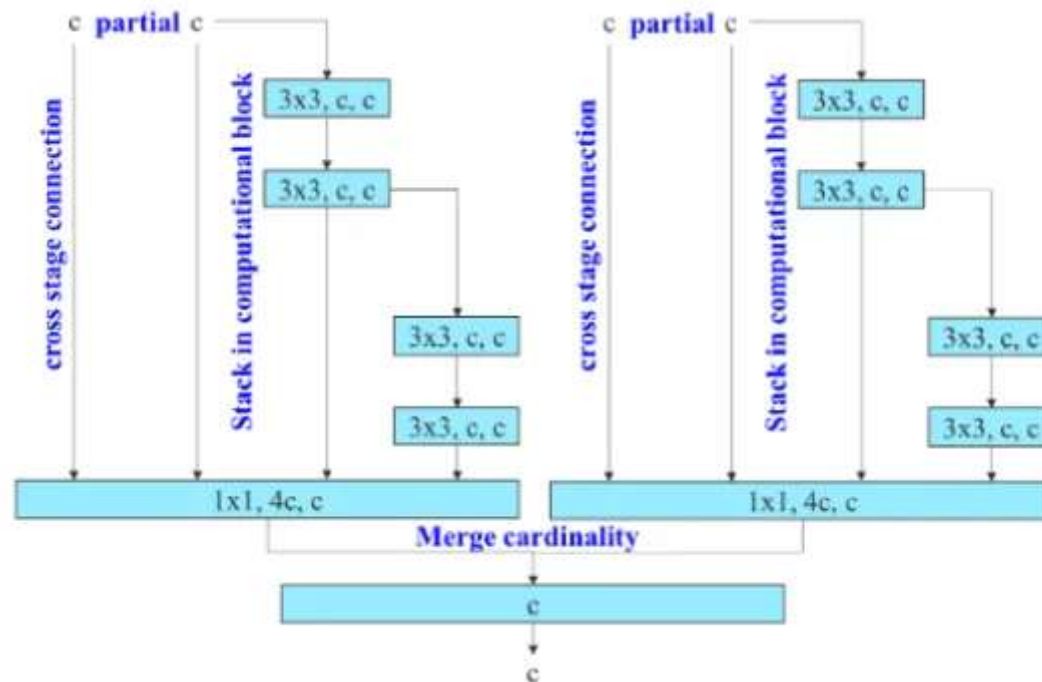


Figure 4: E-ELAN alternative architecture [source](#)

EDIT: Thanks to the reader <https://medium.com/@rares926> for pointing out that at yolov7 only the yolov7-e6e model has E-ELAN, all other variants of yolov7 actually has ELAN. You can verify that at page 16 (Figure A2) in the final version of the paper here <https://github.com/WongKinYiu/yolov7/blob/main/paper/yolov7.pdf>

What is cardinality, you may ask?

In a standard convolutional layer, all input channels are connected to all output channels — so the *cardinality is 1*. However, in group convolutions, the channels are divided into multiple groups, and within each group, the connectivity is restricted. This grouping introduces the notion of cardinality — meaning you have *more than 1 cardinality* when there is group convolution.

How to interpret $3 \times 3, c, c$?

$3 \times 3, c, c$ signifies a convolutional layer that uses a 3×3 filter to convolve over the input data, where the input has 'c' channels, and the output will have 'c' channels.

2.2 Compound Model Scaling

Model scaling refers to width (channel) or depth (number of layers) scaling.

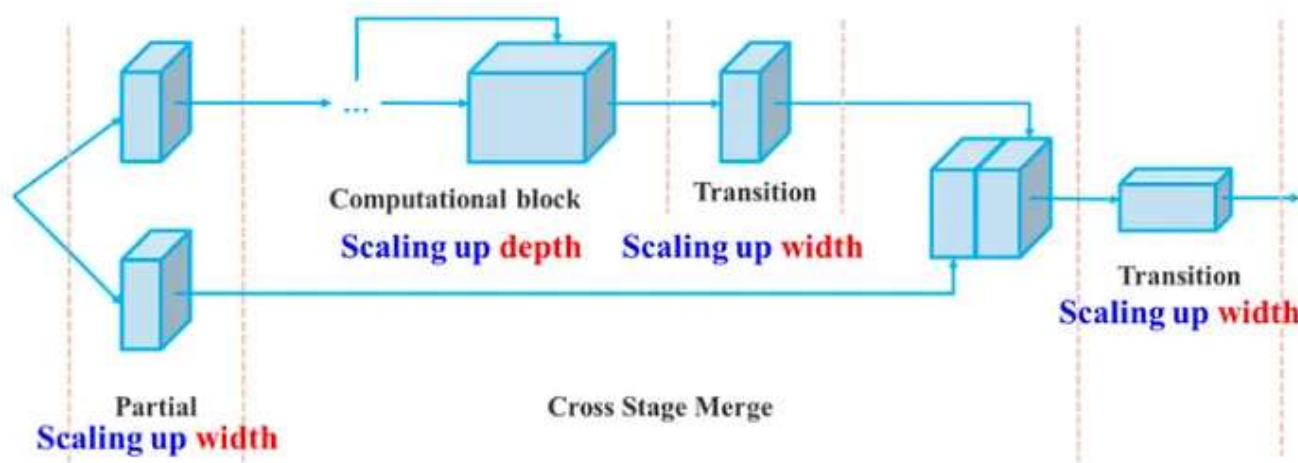


Figure 5: Compound model scaling technique applied in YOLOv7

The authors found that when depth scaling was done in concatenation based models (*computation* block and *transition* block) — it leads to increase in input width in subsequent transition block. Why is that a problem? Because the purpose of transition layer is to do reduction — either channel reduction or dimension reduction of feature map

So to solve this problem- they did depth scaling for only the computational block and width scaling for the transition block.

2.3 Bag of Freebies (BoF)

BoF refers to the techniques that improves the model performance without increasing the training cost. YOLOv7 employed a few different BoFs

- Re-parameterization Convolution
- Label Assignment
- And a few more —

1. re-parameterization on Convolution Batch Normalization,
2. OREPA(Online Convolutional Re-parameterization)
3. YOLO-R

2.3.1 Re-parameterization Convolution (RepConv)

YOLOv7 applied similar RepConv as RepVGG except the identity connection of RepVGG

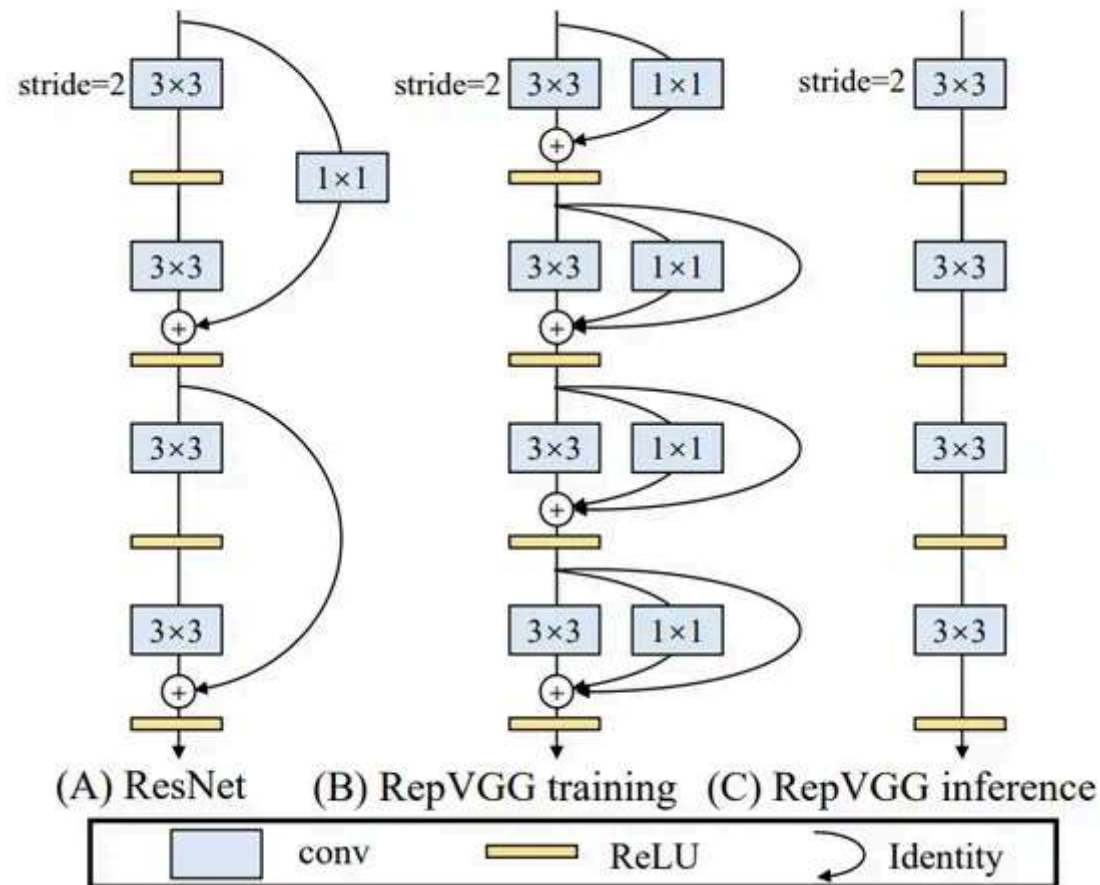
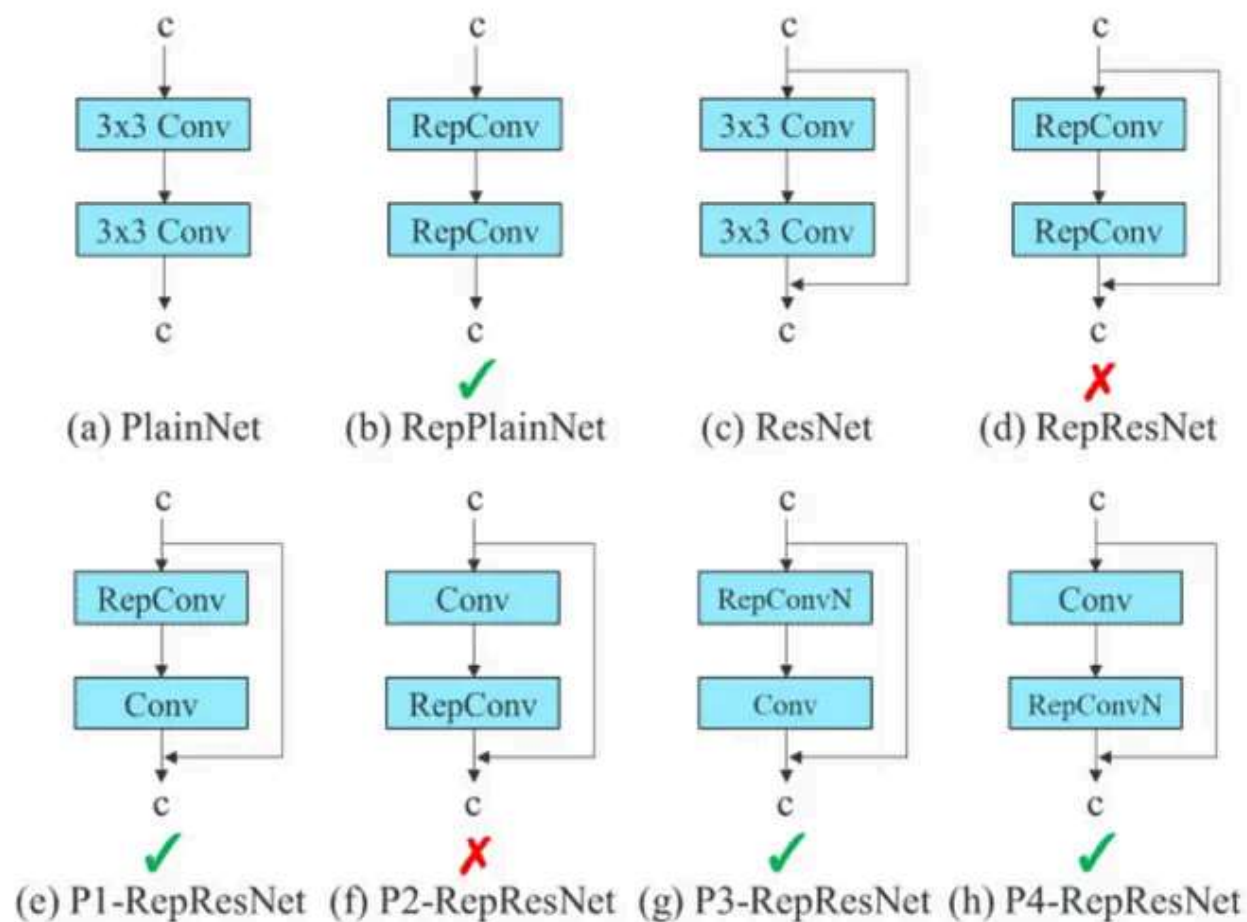


Figure 6: RepVGG

The authors did a reparameterization trial and found out that the identity connection in RepConv destroys the residual in ResNet and the concatenation of DenseNet. This leads to losing diverse gradients from the feature maps. Therefore the authors proposed RepConvN — a RepConv without identity connection

Figure 7: Reparameterization trials by the YOLOv7 authors [source](#)

2.3.2 Varying granularity of label assignment

YOLOv7 borrows the concept of ‘Deep Supervision’ where there are two different *types* of head

- **Lead head** — is responsible for final prediction

- **Auxiliary head** — assists training in previous layers with ‘**assistant loss**’

These heads work with the concept of `label assigner`. A label assigner considers network prediction results together with ground truth and assigns **soft labels**, instead of just hard prediction results.

Yolov7 proposes two types of label assigners

Lead head guided label assigner:

This is calculated based on the prediction result of the lead head and the ground truth, and generate soft label through the optimization process. This set of soft labels will be used as the target training model for both auxiliary head and lead head.

The reason to do this is because lead head has a relatively strong learning capability, so the soft label generated from it should be more representative of the distribution and correlation between the source data and the target.

Furthermore, we can view such learning as a kind of generalized residual learning. By letting the shallower auxiliary head directly learn the

information that lead head has learned, lead head will be more able to focus on learning residual information that has not yet been learned.

Coarse (auxiliary) to fine (lead) label assigner:

This also uses the predicted result of the lead head and the ground truth to generate soft label. However, in the process YOLOv7 generates two different sets of soft label — **coarse label and fine label**,

fine label is the same as the soft label generated by lead head guided label assigner

coarse label is generated by allowing more grids to be treated as positive target by relaxing the constraints of the positive sample assignment process.

The reason for this is that the learning ability of an auxiliary head is not as strong as that of a lead head, and in order to avoid losing the information that needs to be learned, we will focus on optimizing the recall of auxiliary head in the object detection task.

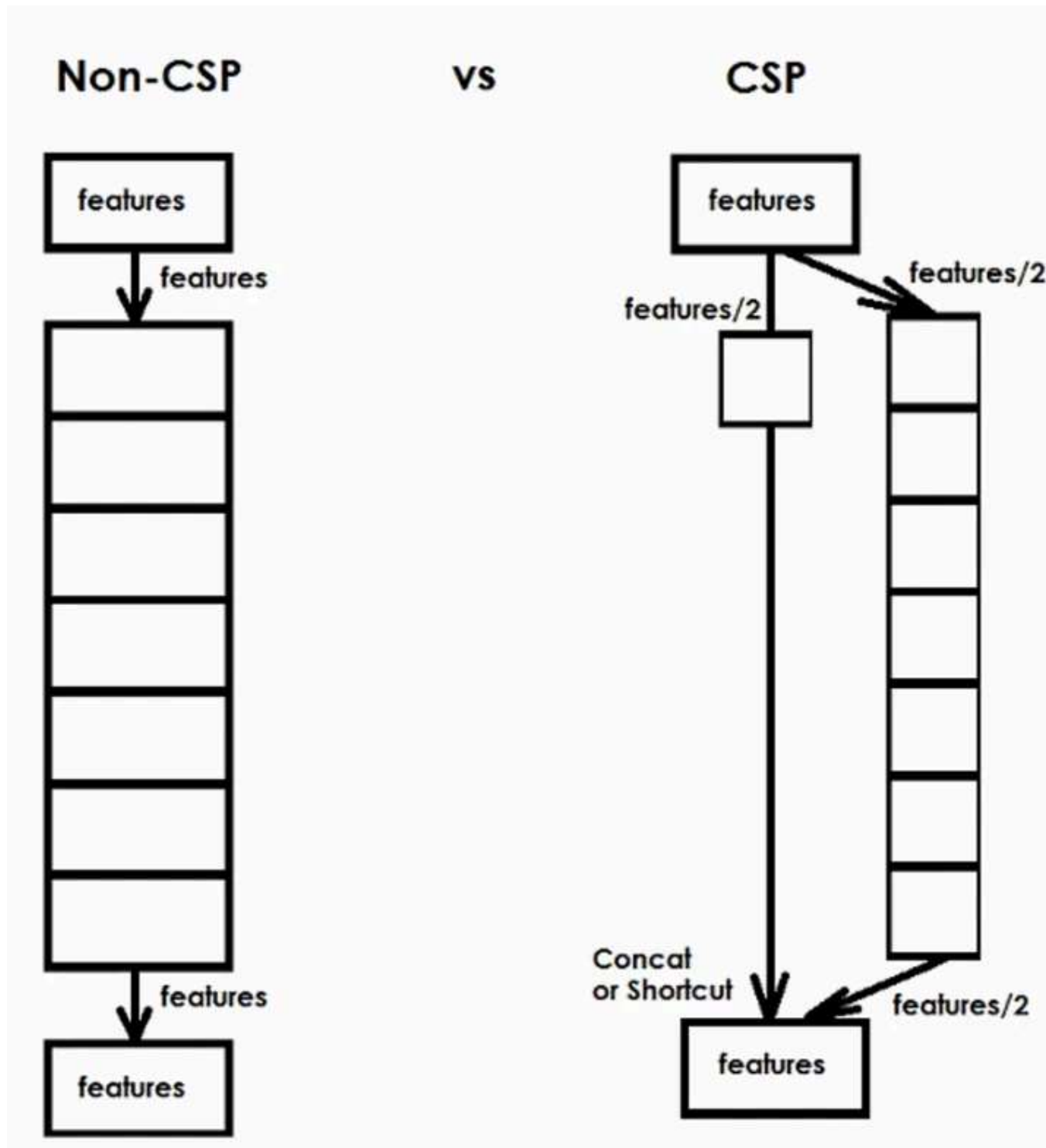
3. Appendix

3.1 Cross Stage Partial Network (CSPNet)

Cross Stage Partial Network is a new way to architect convolutional neural networks that reduces computation for various CNN networks — up to 50%

In a CSP connection:

- Half of the output signal goes along the main path that helps generate more semantic information with a large receptive field.
- The other half of the signal pivots help retain more spatial information with a small receptive field.



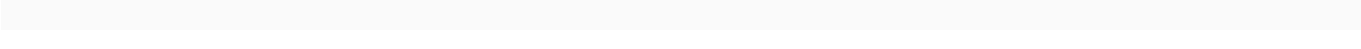


Figure 8: Collected from [source](#).

3.2 Computational block vs transition block

A **computation** block refers to a specific component or unit within the network architecture that performs a set of computational operations. Typically it consists of several layers, such as convolutional layers, activation functions, pooling layers, normalization layers, and possibly other operations specific to the network architecture. These layers work together to extract and refine features from the input data.

A **transition** block refers to a component or layer that is commonly used in architectures like DenseNet. The purpose of a transition block is to reduce the spatial dimensions (width and height) of the feature maps while maintaining or controlling the number of channels.

Typically, a **transition block** is inserted between two dense blocks in a DenseNet architecture. A dense block is a series of connected convolutional layers where each layer receives inputs from all preceding layers within the block. The dense block helps to capture and propagate feature information effectively through the network.

The transition block serves two main purposes:

1. **Dimensionality reduction:** The transition block incorporates layers like 1x1 convolutional layers or pooling layers to decrease the width and height of the feature maps. This reduction helps to downsample the spatial dimensions of the feature maps.
2. **Channel compression:** Along with reducing spatial dimensions, a transition block may also include 1x1 convolutions to adjust the number of channels in the feature maps. This helps in reducing the computational complexity and the number of parameters in subsequent layers.

3.3 How to read the YOLOv7 network architecture?

In YOLO models, the network architectures are specified in a config yaml file. You can find a sample training config file is [here](#). Let's focus on the backbone part of the network

```
# [from, number, module, args],  
[[-1, 1, ReOrg, []], # 0  
 [-1, 1, Conv, [80, 3, 1]], # 1-P1/2  
  
 [-1, 1, DownC, [160]], # 2-P2/4  
 [-1, 1, Conv, [64, 1, 1]],  
 [-2, 1, Conv, [64, 1, 1]],  
 [-1, 1, Conv, [64, 3, 1]],  
 [-1, 1, Conv, [64, 3, 1]],  
 [-1, 1, Conv, [64, 3, 1]],
```

```

[-1, 1, Conv, [64, 3, 1]],
[-1, 1, Conv, [64, 3, 1]],
[-1, 1, Conv, [64, 3, 1]],
[[-1, -3, -5, -7, -8], 1, Concat, [1]],
[-1, 1, Conv, [160, 1, 1]], # 12
[-11, 1, Conv, [64, 1, 1]],
[-12, 1, Conv, [64, 1, 1]],
[-1, 1, Conv, [64, 3, 1]],
[-1, 1, Conv, [64, 3, 1]],
[-1, 1, Conv, [64, 3, 1]],
[-1, 1, Conv, [64, 3, 1]],
[-1, 1, Conv, [64, 3, 1]],
[-1, 1, Conv, [64, 3, 1]],
[[-1, -3, -5, -7, -8], 1, Concat, [1]],
[-1, 1, Conv, [160, 1, 1]], # 22
[[-1, -11], 1, Shortcut, [1]], # 23

```

```

[-1, 1, DownC, [320]], # 24-P3/8
[-1, 1, Conv, [128, 1, 1]],
[-2, 1, Conv, [128, 1, 1]],
[-1, 1, Conv, [128, 3, 1]],
[-1, 1, Conv, [128, 3, 1]],
[-1, 1, Conv, [128, 3, 1]],
[-1, 1, Conv, [128, 3, 1]],
[-1, 1, Conv, [128, 3, 1]],
[-1, 1, Conv, [128, 3, 1]],
[[-1, -3, -5, -7, -8], 1, Concat, [1]],
[-1, 1, Conv, [320, 1, 1]], # 34
[-11, 1, Conv, [128, 1, 1]],
[-12, 1, Conv, [128, 1, 1]],
[-1, 1, Conv, [128, 3, 1]],
[-1, 1, Conv, [128, 3, 1]],
[-1, 1, Conv, [128, 3, 1]],
[-1, 1, Conv, [128, 3, 1]],
[-1, 1, Conv, [128, 3, 1]],
[-1, 1, Conv, [128, 3, 1]],
[[-1, -3, -5, -7, -8], 1, Concat, [1]],

```

```

[-1, 1, Conv, [320, 1, 1]], # 44
[[-1, -11], 1, Shortcut, [1]], # 45

[-1, 1, DownC, [640]], # 46-P4/16
[-1, 1, Conv, [256, 1, 1]],
[-2, 1, Conv, [256, 1, 1]],
[-1, 1, Conv, [256, 3, 1]],
[-1, 1, Conv, [256, 3, 1]],
[-1, 1, Conv, [256, 3, 1]],
[-1, 1, Conv, [256, 3, 1]],
[-1, 1, Conv, [256, 3, 1]],
[-1, 1, Conv, [256, 3, 1]],
[[-1, -3, -5, -7, -8], 1, Concat, [1]],
[-1, 1, Conv, [640, 1, 1]], # 56
[-11, 1, Conv, [256, 1, 1]],
[-12, 1, Conv, [256, 1, 1]],
[-1, 1, Conv, [256, 3, 1]],
[-1, 1, Conv, [256, 3, 1]],
[-1, 1, Conv, [256, 3, 1]],
[-1, 1, Conv, [256, 3, 1]],
[-1, 1, Conv, [256, 3, 1]],
[-1, 1, Conv, [256, 3, 1]],
[[-1, -3, -5, -7, -8], 1, Concat, [1]],
[-1, 1, Conv, [640, 1, 1]], # 66
[[-1, -11], 1, Shortcut, [1]], # 67

[-1, 1, DownC, [960]], # 68-P5/32
[-1, 1, Conv, [384, 1, 1]],
[-2, 1, Conv, [384, 1, 1]],
[-1, 1, Conv, [384, 3, 1]],
[-1, 1, Conv, [384, 3, 1]],
[-1, 1, Conv, [384, 3, 1]],
[-1, 1, Conv, [384, 3, 1]],
[-1, 1, Conv, [384, 3, 1]],
[-1, 1, Conv, [384, 3, 1]],
[[-1, -3, -5, -7, -8], 1, Concat, [1]],
[-1, 1, Conv, [960, 1, 1]], # 78

```

```

[-11, 1, Conv, [384, 1, 1]],
[-12, 1, Conv, [384, 1, 1]],
[-1, 1, Conv, [384, 3, 1]],
[-1, 1, Conv, [384, 3, 1]],
[-1, 1, Conv, [384, 3, 1]],
[-1, 1, Conv, [384, 3, 1]],
[-1, 1, Conv, [384, 3, 1]],
[-1, 1, Conv, [384, 3, 1]],
[[-1, -3, -5, -7, -8], 1, Concat, [1]],
[-1, 1, Conv, [960, 1, 1]], # 88
[[-1, -11], 1, Shortcut, [1]], # 89

[-1, 1, DownC, [1280]], # 90-P6/64
[-1, 1, Conv, [512, 1, 1]],
[-2, 1, Conv, [512, 1, 1]],
[-1, 1, Conv, [512, 3, 1]],
[-1, 1, Conv, [512, 3, 1]],
[-1, 1, Conv, [512, 3, 1]],
[-1, 1, Conv, [512, 3, 1]],
[-1, 1, Conv, [512, 3, 1]],
[-1, 1, Conv, [512, 3, 1]],
[[-1, -3, -5, -7, -8], 1, Concat, [1]],
[-1, 1, Conv, [1280, 1, 1]], # 100
[-11, 1, Conv, [512, 1, 1]],
[-12, 1, Conv, [512, 1, 1]],
[-1, 1, Conv, [512, 3, 1]],
[-1, 1, Conv, [512, 3, 1]],
[-1, 1, Conv, [512, 3, 1]],
[-1, 1, Conv, [512, 3, 1]],
[-1, 1, Conv, [512, 3, 1]],
[-1, 1, Conv, [512, 3, 1]],
[[-1, -3, -5, -7, -8], 1, Concat, [1]],
[-1, 1, Conv, [1280, 1, 1]], # 110
[[-1, -11], 1, Shortcut, [1]], # 111
]

```

Breaking down the architecture above —

1. The backbone network begins with a `ReORG` layer that rearranges the input tensor
2. Following that, there is a convolutional layer (`Conv`) with 80 filters and a kernel size of 3×3 . This layer operates on the input at a stride of 1 and produces output at $1/2$ resolution compared to the input image.
3. Next, a downsampling layer (`DownC`) with 160 filters is applied, reducing the resolution of the feature maps to $1/4$ of the input image.
4. The subsequent layers form a sequence of convolutional layers (`Conv`) with 64 filters and a kernel size of 1×1 or 3×3 . These layers are applied on the feature maps and followed by a concatenation layer (`Concat`) that concatenates the feature maps from the previous layers.
5. This sequence of convolutional and concatenation layers is repeated multiple times.
6. At certain points in the architecture, there are shortcut connections (`Shortcut`). These connections add the feature maps from previous layers to the current feature maps, facilitating the flow of information and enabling the model to capture both low-level and high-level features.

7. The process of downsampling and convolutional operations continues, gradually reducing the resolution of the feature maps and increasing the number of filters.
8. The architecture concludes with a final concatenation layer (Concat) that combines the feature maps from the previous layers.

To understand how E-ELAN is represented in the backbone, you can refer to the issue [here](#).

4. References

1. YOLOv7 paper
<https://github.com/WongKinYiu/yolov7/blob/main/paper/yolov7.pdf>
2. ELAN <https://arxiv.org/pdf/2211.04800.pdf>
3. E-ELAN in the code <https://github.com/WongKinYiu/yolov7/issues/339>
4. <https://www.cameralyze.co/blog/yolov7-architecture-explanation>
5. <https://learnopencv.com/yolov7-object-detection-paper-explanation-and-inference/>

Special thanks to Ishan Khatri for his YOLO presentation at Cohere community.

Artificial Intelligence

Computer Vision

Object Detection

Yolov7



Written by Nahid Alam

565 Followers

Follow



Working on Computer Vision at Meraki Camera Intelligence <https://twitter.com/nahidalam>

More from Nahid Alam

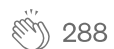


 Nahid Alam in Towards Data Science

Recommender Systems from Learned Embeddings

A Deep Neural Network Approach

🌟 · 6 min read · Jul 12, 2020



288



5



 Nahid Alam in Towards Data Science

Object Detection Neural Network: Building a YOLOX Model on a...

A Comprehensive Guide

🌟 · 7 min read · Jun 21, 2022



92



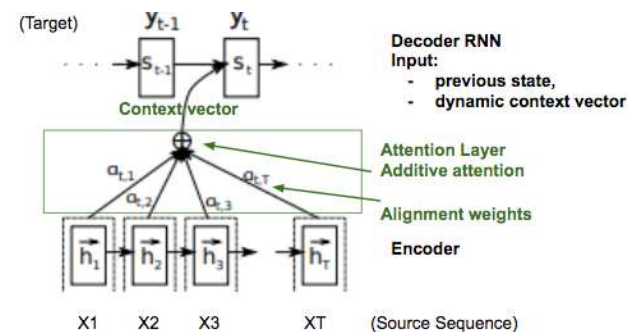
1



 Nahid Alam in Towards Data Science

Building a CycleGAN model with Custom Dataset using Tensorflow 2

And deploying in Hugging Face Space



 Nahid Alam in Towards Data Science

Sequence-to-Sequence Models: Attention Network using...

Part 2: Sequence-to-Sequence models: from RNN to Transformer

★ · 6 min read · Feb 7, 2022



139



★ · 4 min read · Sep 14, 2020



13

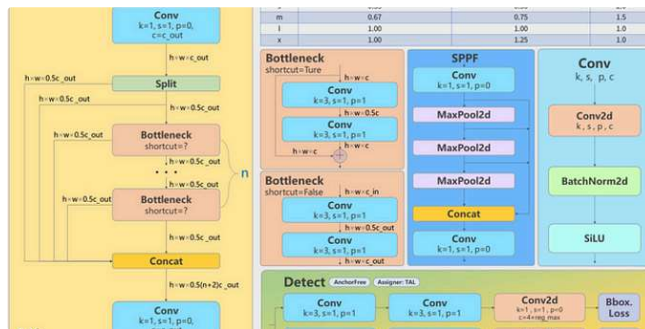


1



See all from Nahid Alam

Recommended from Medium



Juan Pedro



VK

Detailed Explanation of YOLOv8 Architecture—Part 1

YOLO (You Only Look Once) is one of the most popular modules for real-time object...

6 min read · Dec 3, 2023



113



1



YoloV8 Architecture & Cow Counter With Region Based...

Computer vision is an interdisciplinary scientific field that deals with how computer...

8 min read · Nov 21, 2023



116



2



Lists



AI Regulation

6 stories · 431 saves



Generative AI Recommended Reading

52 stories · 988 saves



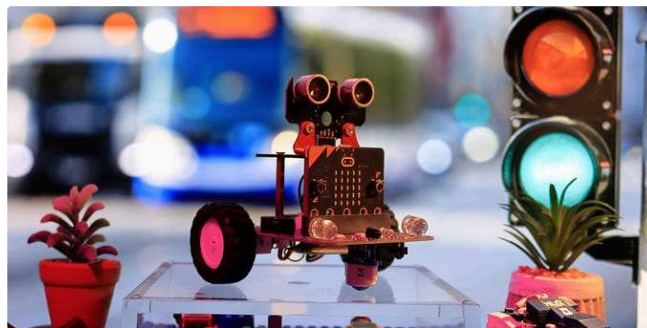
ChatGPT

21 stories · 607 saves



ChatGPT prompts

47 stories · 1497 saves





Fine-tuning YOLOv8

Step-by-step guide for fine-tuning YOLOv8 using your own datasets in Google Colab

🌟 · 3 min read · Feb 7, 2024



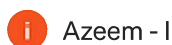
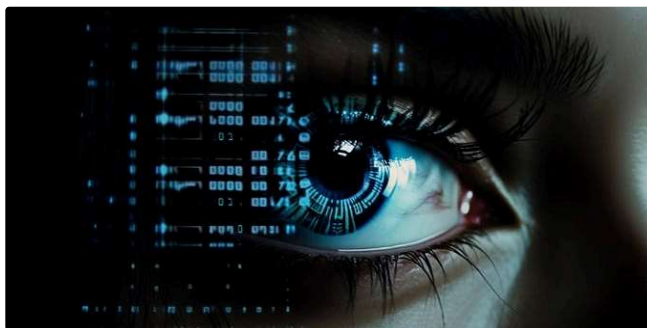
334



3



...



Understanding ResNet Architecture: A Deep Dive into...

Residual Network is a deep Learning model used for computer vision applications. The...

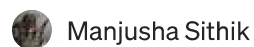
6 min read · Nov 14, 2023



108



...



My Experiments with Yolov5: Almost everything you wa...

Change Scaling related architecture to improve small object detection

8 min read · Dec 25, 2023



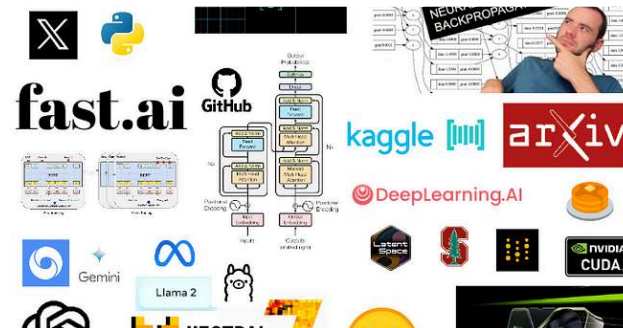
21



1



...



Roadmap to Learn AI in 2024

A free curriculum for hackers and programmers to learn AI

11 min read · Mar 10, 2024



11.1K



114



...

See more recommendations

[Help](#) [Status](#) [About](#) [Careers](#) [Blog](#) [Privacy](#) [Terms](#) [Text to speech](#) [Teams](#)